

ServiceDesk AI

A cloud-native customer support SaaS platform for small businesses

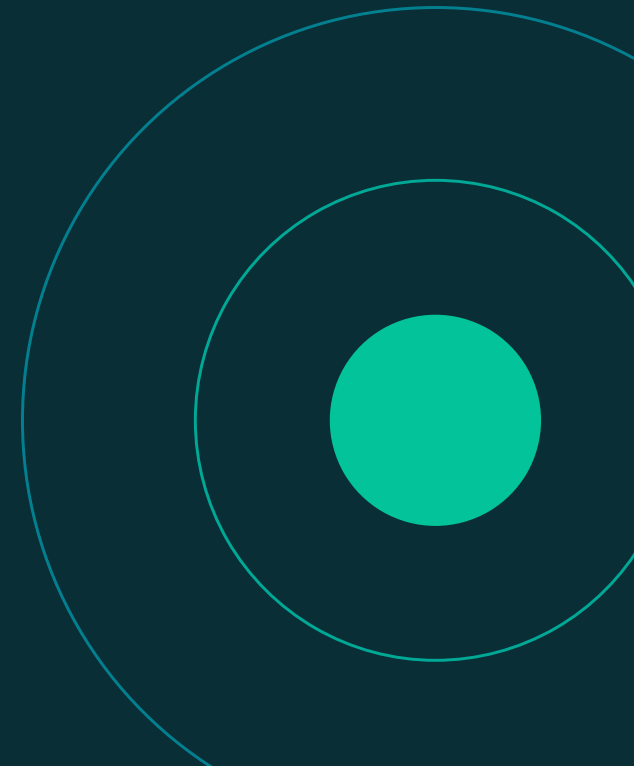
Project 19 • Customer Support SaaS • Review 2 & Final Demonstration • September 2026

Aarush Jagannathan

23BLC1211

Gomatheswar M

23BLC1195



What We Built

A deployed customer support platform: WhatsApp as the intake channel, three roles, a guarded ticket lifecycle, dashboards and reports on live data, and an AI layer that never acts without a human.

64

REST endpoints

across 11 groups

19

Screens

responsive from 375 px

12

Database tables

versioned SQL migrations

161

Automated tests

all passing

Delivered against the nine mandatory requirements



Registration & login



Role-based access



CRUD operations



Dashboard & reports



REST APIs



Responsive UI



GitHub repository



Cloud database



Live deployment

All nine verified on the live system: servicedesk-ai-two.vercel.app on Vercel, API and managed PostgreSQL 16 on Render, WhatsApp through Meta's Cloud API. Total hosting cost: zero.

What We Will Show You

1 Customer

Send a WhatsApp message from a phone — a ticket is raised, classified and acknowledged in a second

2 Isolation

Try to open another customer's ticket — and fail

3 Agent

Prioritised queue, filters, full-text search

4 AI

Generate a draft, edit it, send it — and watch it arrive on the phone

5 Lifecycle

Advance status, then attempt an invalid transition

6 Admin

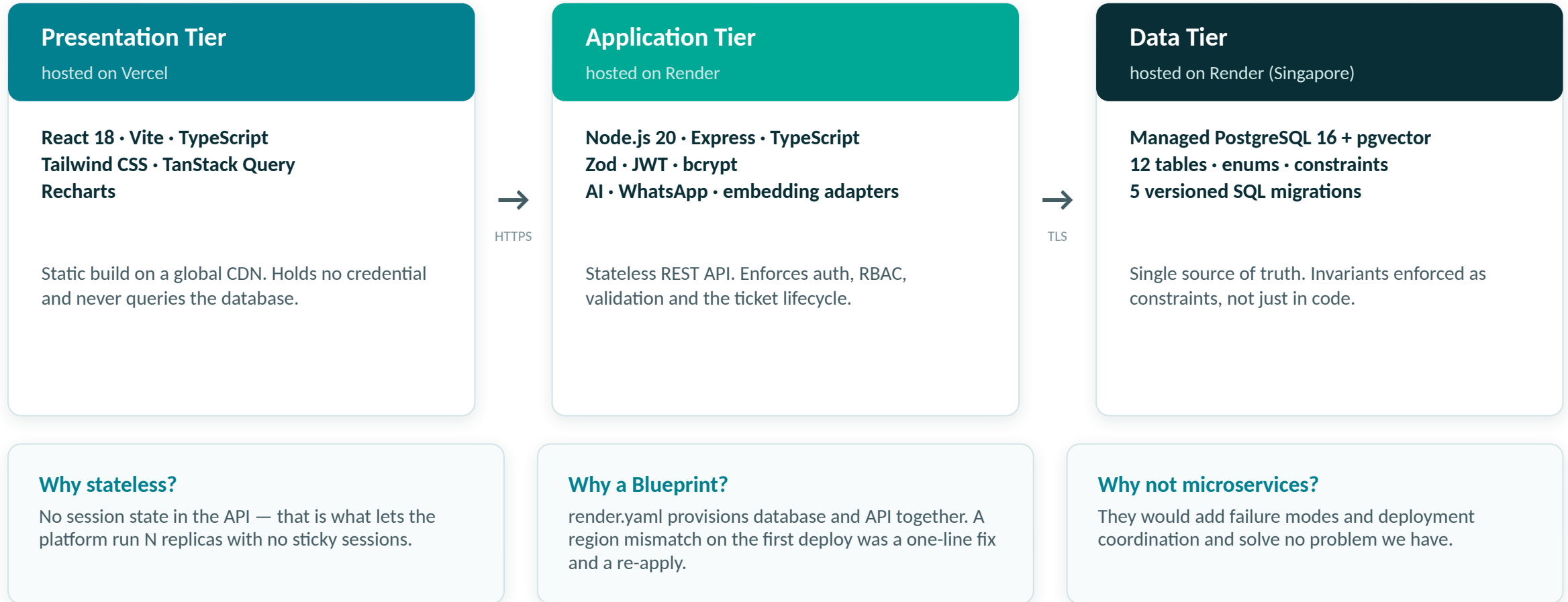
Dashboards, WhatsApp console and audit log computed from the cloud database

Everything is real functionality

- No staged screenshots and no rehearsed data — we create a ticket from a phone in front of you.
- The classification you will see is derived from the words we type, not pre-set.
- Every dashboard figure is a SQL aggregate run at the moment the page loads.

Cloud Architecture

Three independently deployable tiers plus WhatsApp through Meta's Cloud API. The browser talks to two hosts and never to the database or the AI provider.



Database Design

Twelve tables on managed PostgreSQL 16 with pgvector, applied by five versioned SQL migrations on every deploy. The ER diagram is derived from the migrations, so it cannot drift from the real schema.

users

All three roles in one table — role is an attribute, not a different entity

tickets

The core entity. Two foreign keys to users: customer and assigned agent

ticket_events

Append-only timeline: who changed what, from which value, when

notifications

Per-user, cascade on delete · whatsapp_messages — channel log, UNIQUE provider id makes webhooks idempotent

categories

Admin-managed; also the closed vocabulary the classifier may propose from

ticket_messages

The thread. Internal notes filtered in SQL, never in the client

ai_suggestions

Every AI output, with a flag recording whether the fallback produced it

audit_logs

Actor set to NULL on delete · kb_articles / kb_chunks / kb_retrievals — knowledge base with 384-dim vectors

Enums, not lookup tables

Values are fixed by the spec. Declaration order also gives free correct sorting: URGENT above LOW.

Delete rules carry meaning

Customer is RESTRICT — deleting would destroy history. Agent is SET NULL — tickets return to the queue.

Constraints, not just code

resolved_at may only be set when RESOLVED or CLOSED. The invariant holds even if our code is wrong.

Indexes on every filter column, a GIN full-text index for keyword search, and an HNSW vector index for knowledge-base retrieval.

Security and Access Control

Hiding a button is not security. Every claim on this slide is proved by a test that calls the API directly with a valid token for the wrong role.

Two independent authorisation layers

1

Route level

`requireRole()` middleware asks: may this role call this endpoint at all?

2

Record level

The handler asks: does this specific record belong to this caller?

Neither is sufficient alone: middleware cannot know whether ticket X belongs to customer Y.

Passwords

bcrypt at cost 12 — ~250 ms per hash. Salted automatically. Never logged.

Sessions

15-minute access token in memory only; refresh token in an `httpOnly` cookie.

404, not 403

Another customer's ticket returns not-found, so IDs cannot be probed.

Same login error

Unknown email and wrong password give identical responses and timing.

SQL injection

Every query parameterised. The one dynamic value, sort column, is allow-listed.

Secrets

Environment variables only. Nothing secret has ever been committed.

AI Assistance — and Why It Cannot Break Anything

The Project 19 specification does not require AI — only the title contains it. Every capability here is our own addition, so we designed it so that no mandatory requirement depends on it.

Classification

Category, priority, sentiment and a one-line summary on submission

Human control: Agent overrides any field

Draft reply

Generated for the agent to work from

Human control: Agent must insert, edit and send

Summarisation

Condenses a long thread for an agent picking it up cold

Human control: Advisory only

Resolution steps

Suggests troubleshooting steps by category

Human control: Advisory only

Three properties that make it safe

Off the critical path

Classification runs after the ticket row is committed and is never awaited. Ticket creation cannot fail because the AI is down.

Test AI-11

A real fallback, not a stub

A deterministic keyword classifier with category playbooks. The whole system is demonstrable with no API key at all.

Tests AI-01 to AI-07

No path to the customer

Generating a draft creates zero customer-visible messages. Only an agent pressing send does that.

Test AI-12

Why review is mandatory: a model can be confidently wrong about this customer's order, and a human should own what the business says.

Testing

161 automated tests against a real PostgreSQL database, using the same migration files as production — not mocks. They run in CI on every push.

13

Authentication

Registration, sign-in, password storage, tokens, deactivation

26

Authorisation

Every role boundary, called directly against the API

33

Tickets

CRUD, assignment, lifecycle, messages, search, pagination

20

Lifecycle & AI

Transition rules, fallback classifier, failure handling · plus 31 knowledge-base and 38 WhatsApp tests

161 passed · 0 failed · 6 files · 166 s · 19 September 2026

Two results matter most: a total AI failure still creates the ticket, and generating a draft sends nothing.

Defects found and fixed — before submission, not during it

Rate limiter throttled the suite

Found by running the tests

51 failures. The limiter is correct for production; it now skips in test.

A test depended on timing

Found by running the tests

Asserted an exact suggestion count while classification runs in background.

A vector index returned one row where four matched

Found by an integration test

IVFFlat on a near-empty table silently under-returned. Replaced with HNSW; regression test pinned.

The lesson: tests against a real database catch what mocks and green builds cannot.

Compliance — Including What Is Not Done



Complete and verified

Checked by running it, not by assuming

- Registration and login — 13 tests
- Role-based access — 26 tests bypassing the interface
- CRUD and the guarded lifecycle — 33 tests
- Dashboards and 7 reports on live data
- REST API — ~64 endpoints across 11 groups, Swagger UI at /api/docs
- Responsive UI verified at 390, 768 and 1366 px
- GitHub repository with a documented history
- Live on Vercel + Render with managed PostgreSQL; CI green on every push
- WhatsApp two-way channel live through Meta's Cloud API
- All three roles exercised end to end on the live system and from a phone



Not done

Stated plainly, not buried

- No automated frontend tests — verified manually across roles and viewports
- Refresh tokens not individually revocable — deactivation is the mitigation
- File attachments not implemented, so the cloud-storage clause is unmet
- Live LLM path not in the automated suite; its failure handling is
- Meta test number limited to five verified phones until business verification

Every mandatory requirement is complete and verified on the live system. The items on the right are limitations we chose to state rather than hide.

Deployment

Three tiers plus messaging, deployed from a version-controlled Blueprint. Every step below was performed and verified; the problems met are in the report.

1

Database

Render PostgreSQL 16

Provisioned by render.yaml in Singapore with pgvector; migrations run on every start; seeded once

done

2

API

Render

Web service from the same Blueprint: `npm ci --include=dev, migrate && start, /api/health`, generated JWT secrets

done

3

Frontend

Vercel

`servicedesk-ai-two.vercel.app` — root frontend, Vite preset, one public variable: the API base URL

done

4

WhatsApp

Render

Meta WhatsApp Cloud API: signed webhook to `/api/whatsapp/webhook`, app subscribed to the business account

done

Free-tier reality

The free API instance sleeps after 15 minutes idle. An UptimeRobot ping to `/api/health` every 5 minutes keeps it awake for the demonstration.

The credential check

The Vercel project holds exactly one variable, the API base URL. No AI key, WhatsApp token or database credential ever reaches the browser.

Verified live: health, registration, cookie behaviour, ticket creation, AI draft, dashboards, Swagger, mobile layout, and a WhatsApp round trip from a phone in about one second.

Future Enhancements

Production WhatsApp number

Meta business verification lifts the five-recipient limit of the test number and enables templates outside the 24-hour window

Low

File attachments

Screenshots make technical issues far faster to diagnose; also satisfies the cloud-storage clause

Low-moderate

SLA targets & escalation

We measure response and resolution time but do not act on it. This turns measurement into management

Moderate

Semantic embeddings

The knowledge base ships with a lexical fallback embedding; semantic embeddings by default would lift retrieval hit@3 above today's 61.5%

Moderate

Real-time updates

Replaces the 30-second polling and removes the window where two agents act on stale state

Moderate

Multi-tenancy

One deployment serving many businesses. Would extend the same query-scoping mechanism we already use for roles

High

Multi-tenancy is largest — but the shared query builder that scopes by role today is exactly where a tenant filter would go.

Thank you

Questions are welcome — both of us are prepared to answer, and to modify the code.

Project 19

ServiceDesk AI · Customer Support SaaS

Team

Aarush Jagannathan 23BLC1211 · Gomatheswar M 23BLC1195

Guide

Dr. M Dinakaran

Repository

github.com/ItzGOLLY/servicedesk-ai

Tests

161 passed · 0 failed · against real PostgreSQL · CI green

Live URL

servicedesk-ai-two.vercel.app · [API servicedesk-api-ss2d.onrender.com/api](https://api.servicedesk-api-ss2d.onrender.com/api)